

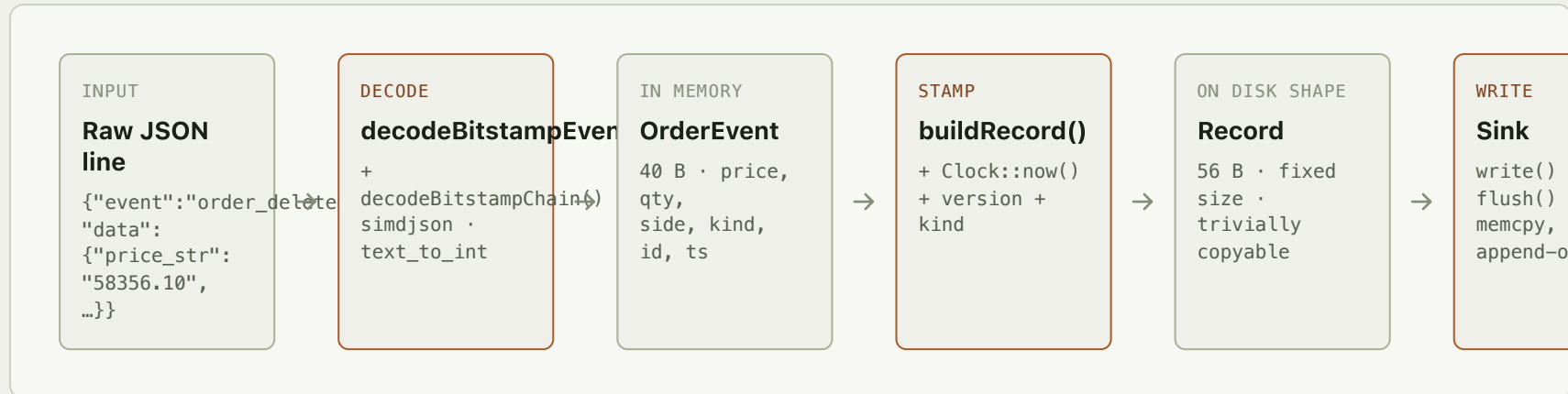
How a market event becomes 56 bytes on disk

A walk through the whole capture path built this session — the Sink, the Record format and its v1→v2 change, and gap detection — in the order data actually moves through it, with a diagram for each stage.

RECORD SIZE	FORMAT	REAL CAPTURE	TESTS
56 B	v1 → v2	1,433 records	80 passing

01 The whole path, end to end

Before zooming into the Sink, here is everywhere a piece of data goes between arriving as text and sitting on disk as bytes. Every box below is real code you can open.



This whole row runs once per line, inside `runRecorder()`. Nothing here allocates on the heap per message — `OrderEvent` and `Record` are plain, fixed-size, trivially-copyable structs, which is what makes the last step a straight `memcpy` instead of a serializer.

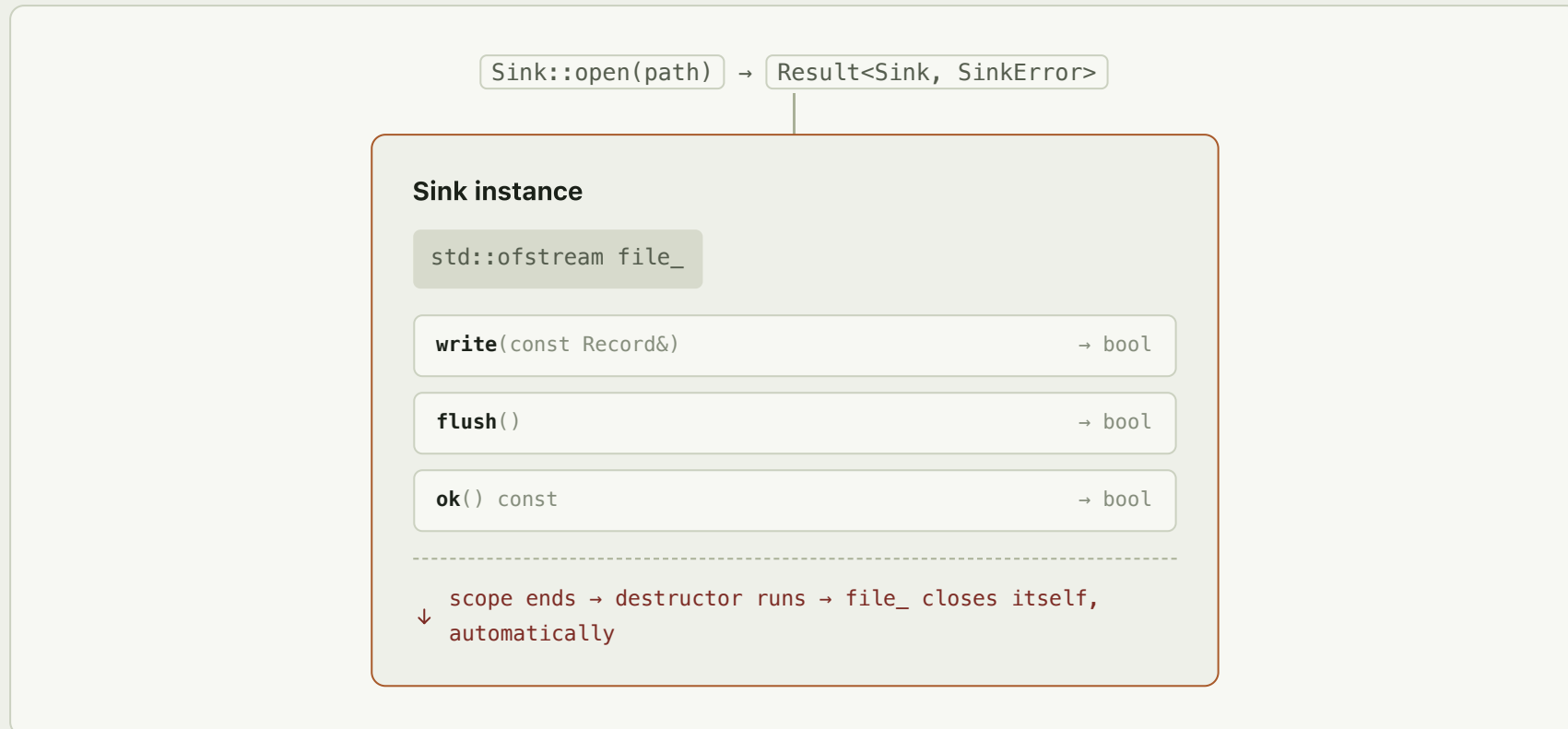
WHY A SEPARATE ORDEREVENT AND RECORD

`OrderEvent` is the in-memory shape the future order book will read. `Record` is the on-disk shape. Keeping them distinct means a change to one for in-memory reasons — say, reordering fields for cache behaviour — can never silently change what a capture file means. `buildRecord()` is the one place that bridges them.

"Sink" is the standard term — spdlog and Serilog both use it — for the destination an event stream is written to. Here it is one small class wrapping one `std::ofstream`.

Sink's shape: one file handle, three operations, RAII lifetime

FIG. 02



There is no default-constructed, unopened `Sink`. The constructor is private, so the only way to get one is through `open()` — meaning a `Sink` that exists in your code is always one that opened successfully. `Sink(Sink&&)` is allowed, copying is deleted: two `Sink` objects appending through the same handle has no sensible meaning.

- **open(path)** — creates the file if absent, opens with `std::ios::app | std::ios::binary`. Failure returns `SinkError::cannot_open` instead of throwing, per this project's no-exceptions convention.

- **write(record)** — appends the record's raw bytes to the stream's own buffer. Returns `true` if accepted — that is *not* a durability guarantee, only that the bytes reached the stream.
- **flush()** — pushes the buffered bytes to the operating system. This is the actual durability boundary this project draws.
- **ok()** — reports whether the underlying stream is still healthy. Once one write fails, the stream is permanently failed; every later write on that same Sink fails too.

WHY APPEND, NOT OVERWRITE

Opened with `std::ios::app`, which on POSIX maps to `O_APPEND`: every write lands at the current end of the file no matter what wrote there before. That single-write atomicity is the same property real append-only logs are built on — syslog, journald, and a database's write-ahead log all rely on it. It is also why running the recorder twice at the same output path silently concatenates two sessions — which is why `recorder_main.cpp` now refuses to open an existing output path rather than trust the caller to remember.

WHY FLUSH AFTER EVERY RECORD

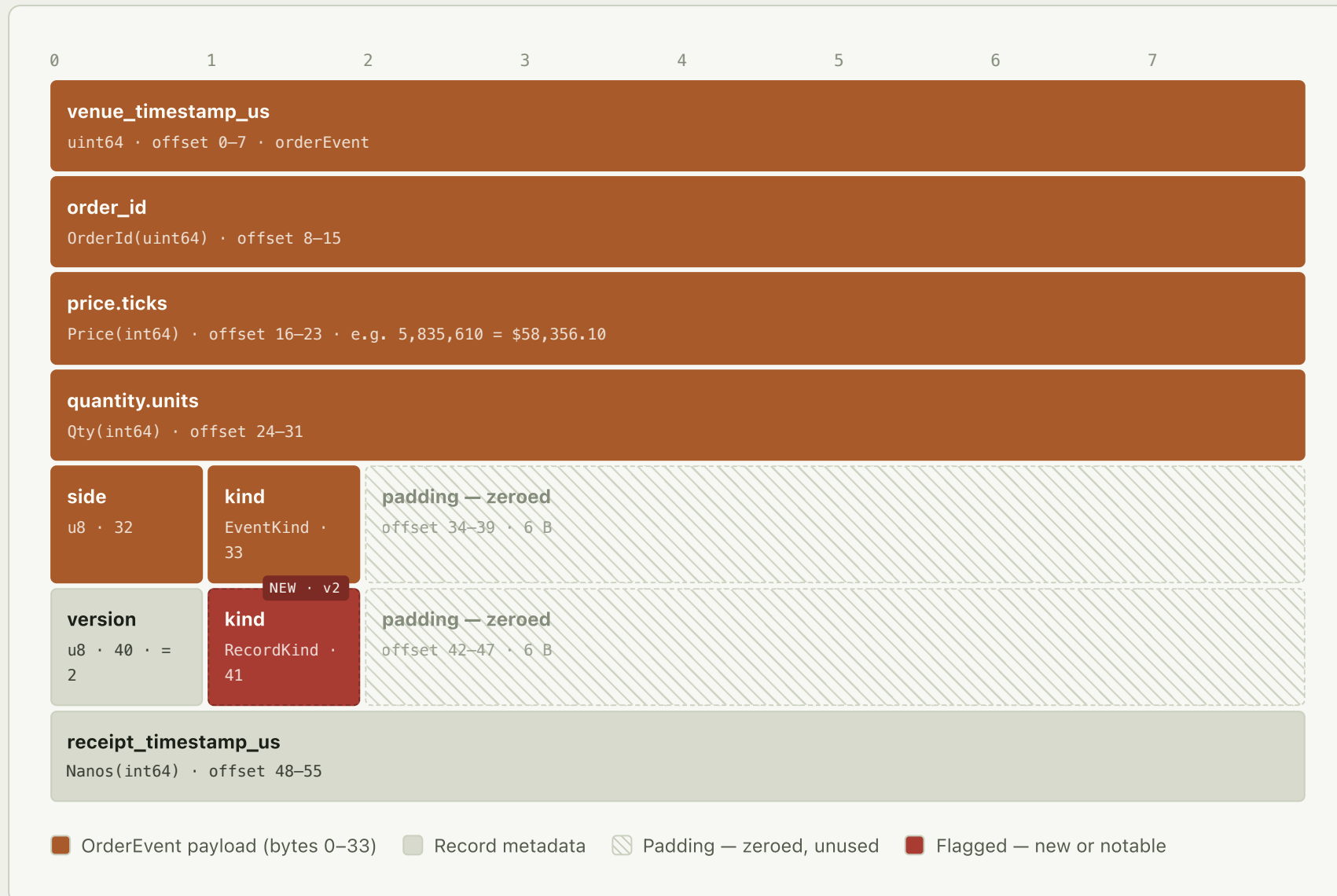
Flushing costs a syscall per event; batching flushes is faster. The choice made here is flush-every-write, which makes the loss window on an unclean shutdown exactly zero records. Nothing in `Sink` enforces this — it holds no flush policy of its own, the same way it does not guess at decimal scale. The caller, `runRecorder()`, decides.

03 **Record: the 56 bytes themselves**

This is the part most worth seeing rather than reading about. Every field below is real, in the order it is actually declared, at the offset the compiler actually places it.

Record v2 memory map — 56 bytes, 7 rows of 8

FIG. 03



Rows line up with 8-byte boundaries because every multi-byte field here is a 64-bit integer, which the compiler must 8-byte-align — that alignment requirement is what produces the padding you see, not a deliberate gap left for later. Bytes 34–39 and 42–47 were originally just wasted space; byte 41 is the one padding byte this session put to use.

v1	byte 41 · padding, unspecified in general — this codebase zeroes it	0x00
v2	byte 41 · RecordKind::order_event	0x00

Same byte, same value — only the meaning changed. That only holds because an earlier fix in this project made `buildRecord()` zero every padding byte before writing (`memset`, not `Record record{}`, which zeroes members but leaves padding untouched — verified by building both ways). A v1 writer that left padding as whatever was on the stack would need a real converter here, not a free reinterpretation.

WHERE THIS BROKE, BRIEFLY

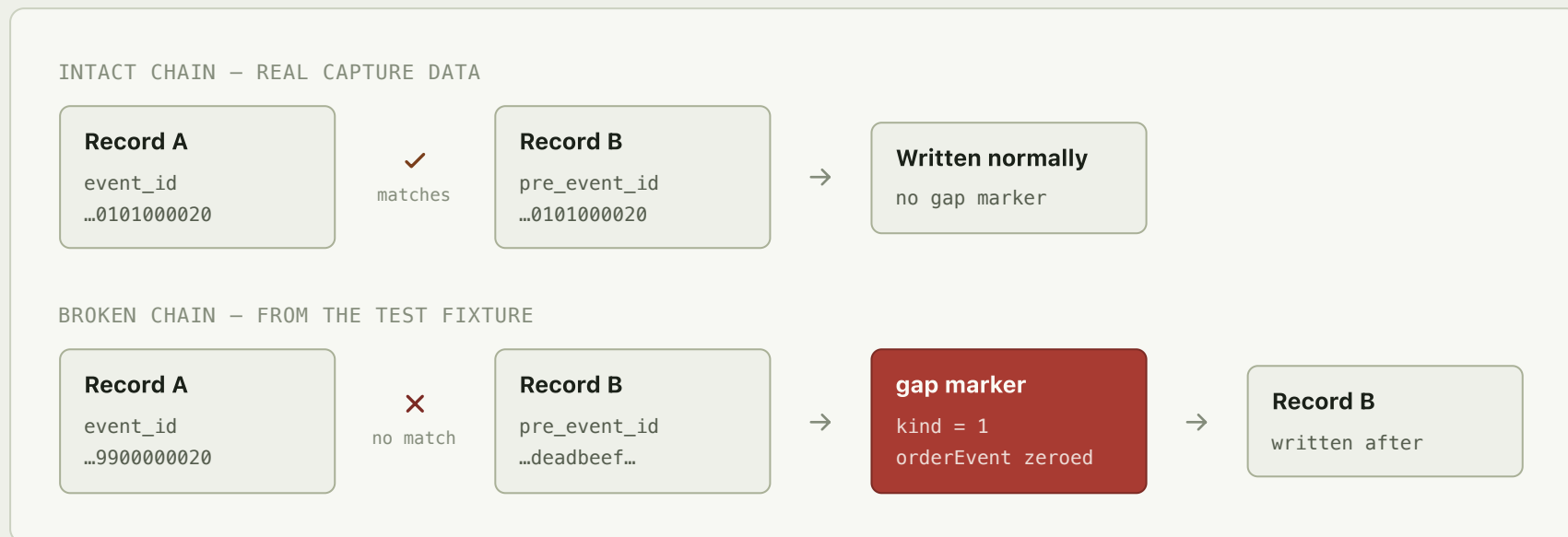
Zeroing with `std::memset(&record, 0, sizeof(record))` compiled fine under clang but failed CI under GCC: `Record` counts as a non-trivially-*default-constructible* type (because `OrderEvent` has default member initialisers), which trips `-Wclass-memaccess` under `-Werror`. It is a false positive — every member is assigned immediately after, and `Record` is trivially *copyable*, which is the property that actually matters — but GCC cannot tell that from a plain pointer. Casting to `void*` is the standard way to say "this is deliberate." Local builds are clang-only, so this only surfaced once CI ran GCC — the reason CI now builds both.

04 Gap detection

Bitstamp does not number its messages. Each one names its own `event_id` and the `pre_event_id` of the message that should have preceded it — so continuity is a chain you verify, not a counter you check.

Matching the chain, and what happens when it breaks

FIG. 05



The gap marker is written *before* the event that revealed the break — its position in the file is what tells a later replay exactly where continuity was lost. The `deadbeef` id above is a genuine test fixture value, chosen because it can never collide with a real UUID.

- A gap proves data was lost but never says *how much* — Bitstamp chains by id, not by count, so there is no arithmetic gap size (ADR 0006).

- The first order event in a capture has no predecessor. Without an explicit guard for that, every capture would open with a false gap.
- Protocol messages like `bts:subscription_succeeded` carry no chain ids at all, and are never treated as breaking continuity.
- An unreadable chain is not treated as a gap either — absence of evidence is not evidence of loss.

WHY IN THE STREAM, NOT A SIDECAR FILE

A gap marker could instead live in a separate file listing breaks. That makes correctness *opt-in*: the one replay that forgets to read the sidecar produces a confidently wrong backtest with no visible symptom. Putting the marker inside the record stream itself means a reader cannot miss it without deliberately ignoring record kinds. Kafka's control records and a database's WAL checkpoint records take the same approach.

05 Making the loop testable

The capture loop originally lived entirely inside `main()`. That is a real problem: there is no `main()` in a test binary, so the only way to check it was running the program by hand and reading numbers off the terminal.



`runRecorder(std::istream&, Sink&, InstrumentSpec, Clock)` takes streams, not paths. That is the whole trick: a test builds a `std::istreamstream` from a literal string and points a real `Sink` at a temp file, then calls the exact same function `main()` calls — no filesystem assumptions, no network, no argv.

This unlocked the golden test the plan had called for since Slice 1 opened: run the full 1,434-line real capture through `runRecorder()` and assert the exact counts, the exact file size, and zero non-zero padding across every record — as a test that runs in CI on every push, not a command run by hand.

06 Where this lives in the repo

Every box in the diagrams above maps to one real file.

HEADER	SOURCE	WHAT IT HOLDS
telemetry/sink.hpp	sink.cpp	Sink — RAll append-only writer
telemetry/record.hpp	record.cpp	Record, RecordKind, buildRecord(), buildGapRecord()
telemetry/recorder.hpp	recorder.cpp	runRecorder() — the testable loop, chain tracking
feed/bitstamp_decoder.hpp	bitstamp_decoder.cpp	decodeBitstampEvent(), ChainLink, decodeBitstampChain()
feed/events.hpp	—	OrderEvent, EventKind
—	apps/recorder_main.cpp	Thin shell: argv, path safety, reporting
—	tests/unit/test_recorder.cpp	12 tests, incl. the golden round-trip test

Verified against the real 1,434-line Bitstamp capture and an 80-test suite, green under

both g++-15 and clang.